

What is binary search? #flashcard

Binary search is an algorithm that locates a target value within a sorted collection by repeatedly dividing the search space in half, comparing the target to the middle element, and discarding the half that cannot contain the answer. It employs a divide-and-conquer strategy using only three pointers—low, high, and mid—to maintain minimal memory overhead. For instance, finding the value 7 in [2, 3, 5, 7, 8, 10, 12] succeeds on the first comparison when the middle element is checked. Why it matters: Binary search achieves $O(\log n)$ efficiency, making it vastly superior to linear search's $O(n)$ for large datasets, which is critical for applications handling millions or billions of records.

What is the precondition that must be satisfied before applying binary search? #flashcard

The input array or collection must be sorted in either ascending or descending order before binary search can be applied; any deviation from this requirement produces undefined or incorrect results. The algorithm relies on the sorted order to make informed decisions about which half of the remaining search space to eliminate at each step. For example, searching for 10 in the sorted array [1, 3, 5, 7, 10, 14, 18] allows the algorithm to confidently compare the middle element and decide whether to search left or right. Why it matters: Skipping the sort verification is a silent killer—binary search will not raise an error but will return wrong answers, making data validation essential before deployment.

What are the three possible outcomes when comparing the target to the middle element? #flashcard

At each iteration, the algorithm checks whether the middle element equals the target (found—return the index), is less than the target (move the low pointer rightward to search the upper half), or is greater than the target (move the high pointer leftward to search the lower half). These three conditions are exhaustive and mutually exclusive, ensuring the search always makes progress toward either finding the target or confirming its absence. When searching for 10 in [1, 3, 5, 7, 10, 14, 18], if the middle element is 7 (too small), the algorithm advances low; if it then checks 14 (too large), it retreats high; finally, 10 matches and is returned. Why it matters: Understanding these three cases is fundamental to implementing or debugging binary search correctly and predicting which direction the search will navigate.

How does the iterative implementation of binary search differ from the recursive implementation in terms of space usage? #flashcard

The iterative approach uses a while loop to repeatedly adjust the low and high pointers, consuming only $O(1)$ constant space regardless of array size, while the recursive approach calls itself with updated bounds and accumulates $O(\log n)$ stack frames on the call stack. Both achieve the same $O(\log n)$ time complexity because the search space halves each iteration or recursive call. Python's iterative implementation using a simple while loop with pointer adjustments is preferred in production systems because it avoids the overhead of function call overhead and stack memory. Why it matters: Choosing iterative over recursive binary search is especially critical in memory-constrained environments or when handling extremely large datasets, as stack overflow can crash recursive implementations.

What is the time complexity of binary search, and how does it scale compared to linear search? #flashcard

Binary search operates in $O(\log n)$ time for average and worst cases, meaning that even an array of one billion elements requires at most approximately 30 comparisons to find any target or confirm its absence. In stark contrast, linear search requires $O(n)$ time, potentially scanning all elements—up to 1,000 comparisons for 1,000 items versus only 10 for binary search. For an array of 1,000 elements, binary search needs at most 10 comparisons while linear search might require up to 1,000 in the worst case. Why it matters: This logarithmic efficiency makes binary search the algorithm of choice for large-scale data retrieval in databases, search engines, and any system where query speed directly impacts user experience.

What is the space complexity of binary search, and how does it differ between iterative and recursive implementations? #flashcard

Iterative binary search maintains only three pointers and uses $O(1)$ constant space, making it highly efficient regardless of input size, whereas recursive binary search consumes $O(\log n)$ space due to the accumulation of function call stack frames during recursion. Both implementations achieve identical time complexity of $O(\log n)$, so the space distinction is the primary trade-off between them. In a recursive search on a million-element array, approximately 20 stack frames would accumulate before returning, whereas iteration never exceeds three active variable references. Why it matters: For embedded systems, real-time applications, or environments with tight memory budgets, the $O(1)$ space guarantee of iterative binary search makes it the preferred implementation strategy.

How does the integer overflow pitfall occur in binary search, and what is the safe formula to prevent it? #flashcard

In languages like Java or C with fixed-width integers, computing $\text{mid} = (\text{low} + \text{high}) / 2$ can overflow when both low and high are large values near the integer limit; for instance, with $\text{low}=2,000,000,000$ and $\text{high}=2,100,000,000$, their sum exceeds the maximum 32-bit signed integer value. The safe alternative is to use $\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$, which computes the midpoint without ever summing values that might overflow. This bug is silent—the program does not crash but produces an incorrect midpoint and searches the wrong half of the array. Why it matters: Integer overflow in binary search has been responsible for real production bugs in major systems, making the safe midpoint formula a non-negotiable best practice in any language with bounded integer types.

How does binary search handle duplicate elements in the array? #flashcard

Standard binary search may return any index of a matching element when duplicates exist, providing no guarantee of which occurrence is found—first, last, or middle. To overcome this limitation, modified versions of binary search are employed: one variant finds the leftmost (first) occurrence by continuing to search left even after finding a match, while another finds the rightmost (last) occurrence by continuing right. For example, in $[1, 5, 5, 5, 9]$, standard binary search might return index 2, but the modified left-biased version would return index 1. Why it matters: Many real-world applications like database range queries or coordinate-based searches require finding all duplicates or their boundaries, making these modified variants essential tools in a programmer's toolkit.

What must you check before applying binary search to prevent silent failures? #flashcard

You must verify that the input array is sorted (ascending or descending) and guard against edge cases like empty arrays with an initial length check before beginning the search. Additionally, you should validate that the loop condition correctly uses $\leq$ rather than $<$ to avoid off-by-one errors that skip elements. For example, an empty array should return -1 immediately rather than proceeding through the loop, and an unsorted array produces wrong answers without raising any exception. Why it matters: Binary search fails silently on invalid inputs—no error is thrown, making input validation a critical defensive programming practice to catch bugs during testing rather than in production.

How does binary search work in the context of database indexing? #flashcard

Databases extend binary search logic through B-tree structures, which organize records hierarchically so that lookups can narrow down to the target record in logarithmic time rather than scanning all rows. The B-tree maintains sorted order at each level, enabling the database engine to apply binary search principles across millions of records stored on disk. When you query a database index for a specific customer ID, the database engine effectively performs a series of binary searches, first in the root node, then in child nodes, until reaching the leaf containing the exact record. Why it matters: Without binary search and B-tree indexing, databases would degrade to full table scans, making modern high-performance systems handling petabytes of data impossible to achieve.

How does binary search apply to finding square roots and solving monotonic equations? #flashcard

Binary search can be adapted to find numerical solutions by treating the search space as a continuous range rather than discrete array indices, checking a midpoint value against the target condition, and narrowing the range based on whether the midpoint is too small or too large. For square root calculation, you can search the range $[0, n]$ to find the value x where x^2 equals the target number, using the midpoint to test whether x^2 is below, equal to, or above the target. Similarly, for monotonic functions (functions that always increase or always decrease), binary search efficiently finds the value at which the function crosses a threshold. Why it matters: This “binary search on the answer” technique is a powerful competitive programming strategy that solves problems in $O(\log n)$ time that might otherwise require $O(n)$ or slower approaches.

How is binary search utilized in Git’s bisect feature? #flashcard

Git bisect employs binary search to locate the specific commit that introduced a bug by repeatedly dividing the commit history in half and asking developers whether a tested midpoint commit exhibits the bug. If the midpoint commit is buggy, the search narrows to earlier commits; if it is clean, the search narrows to later commits, eventually pinpointing the exact commit responsible. For a repository with 1,000 commits, this approach finds the culprit in approximately 10 bisection steps instead of manually checking commits linearly. Why it matters: This application demonstrates how binary search extends beyond data retrieval to version control systems, enabling developers to rapidly diagnose regressions and maintain code quality in large projects.

What is the difference between binary search and linear search in terms of efficiency and suitability? #flashcard

Linear search examines elements sequentially from start to end, taking $O(n)$ time in the worst case, and requires no precondition on array order, making it suitable for unsorted or small datasets. Binary search requires a sorted array and achieves $O(\log n)$ time, making it dramatically faster for large datasets but unsuitable for unsorted data without prior sorting. For a million-element array, linear search might require 1,000,000 comparisons while binary search needs only 20, but if you must sort first, the overhead may not justify binary search for tiny datasets. Why it matters: Choosing between these algorithms requires understanding both your data's size and whether it is already sorted—premature optimization with binary search on unsorted data is worse than using linear search, whereas ignoring binary search on massive sorted datasets is a critical performance mistake.

What is the difference between the best case and worst case time complexity of binary search? #flashcard

The best case occurs when the target is the first middle element checked, requiring only $O(1)$ time with a single comparison, whereas the worst case occurs when the target is absent or located at a leaf of the search tree, requiring $O(\log n)$ comparisons. The average case also lands at $O(\log n)$ because the algorithm consistently halves the search space regardless of target position. For a 1,000-element array, best case takes 1 comparison, but worst case takes up to 10 comparisons before confirming the target is absent. Why it matters: Understanding this distinction helps set realistic performance expectations—while $O(\log n)$ is guaranteed even in the worst case, real-world binary search often terminates faster, and recognizing this can help optimize systems by caching frequently accessed elements that benefit from best-case behavior.

How do Python's `bisect` module, Java's `Arrays.binarySearch()`, and C++'s `std::binary_search` reflect the ubiquity of binary search in standard libraries? #flashcard

Each major programming language provides built-in binary search implementations in their standard libraries—Python's `bisect` for sorted list insertion and searching, Java's `Arrays.binarySearch()` for arrays, and C++'s `std::binary_search` for containers—demonstrating that binary search is considered a fundamental operation. These library implementations are optimized, extensively tested, and handle edge cases, making them preferable to writing custom versions unless you have specialized requirements like finding first or last occurrence. For example, `bisect.bisect_left([1,3,5,7,9], 6)` returns index 3 (the insertion point) in $O(\log n)$ time without requiring manual implementation. Why it matters: Relying on standard library implementations reduces bugs, improves code readability, and ensures portability across platforms, making it a best practice to prefer battle-tested library functions over custom algorithms whenever possible.